

CSE 160 (Fall 2020) – Homework 3

Due: Nov 9 at 11:59pm PT on Canvas.

Total: 10 points.

Colors

1. Fill in the blanks to convert colors between RGB, HSV, and CMY models

Solution.

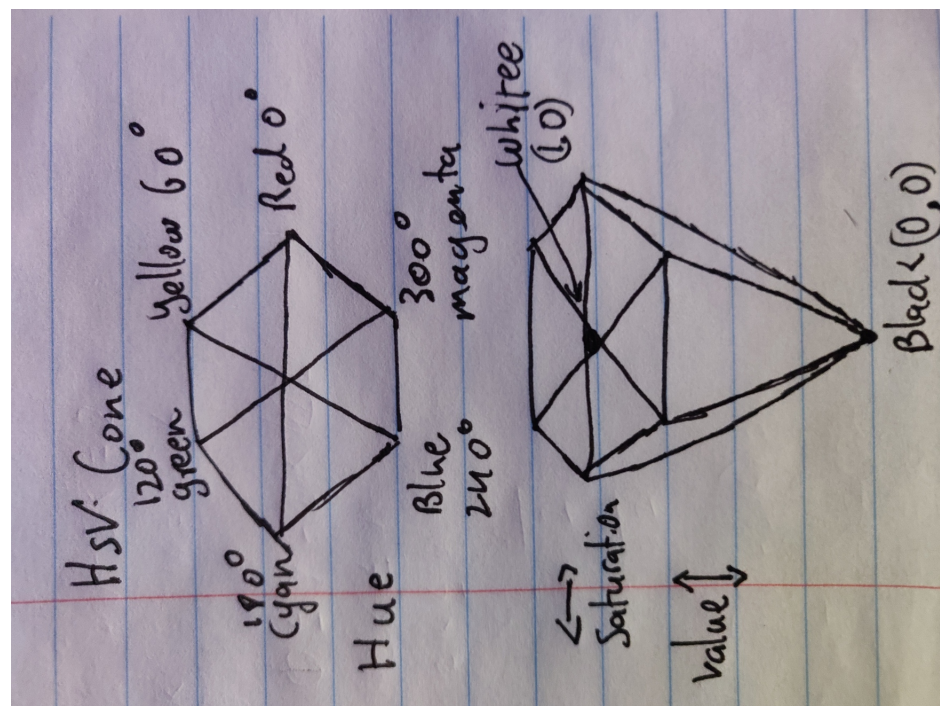
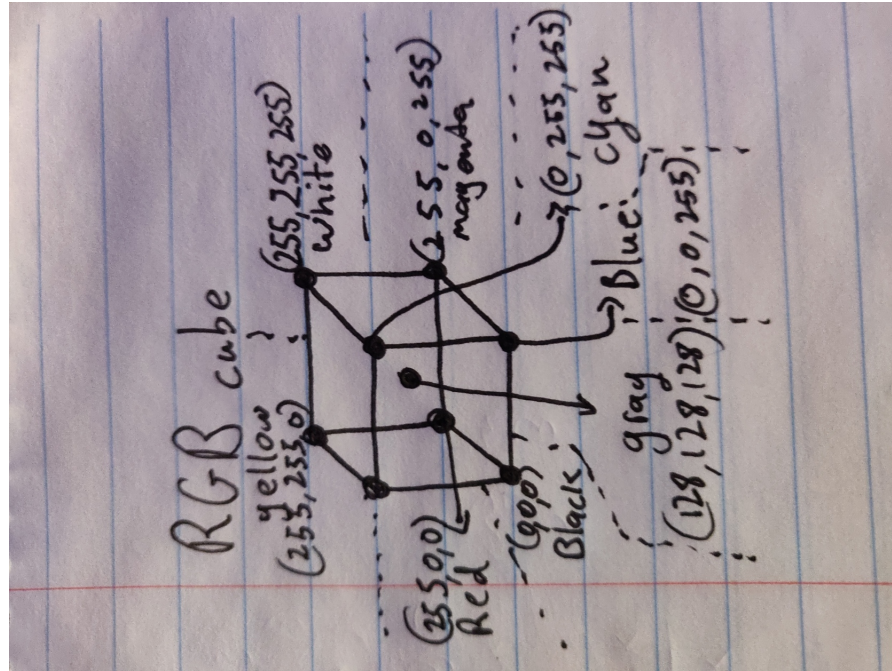
	Red	Green	Blue	Hue°	Saturation%	Value %	Cyan	Magenta	Yellow
(a)	0	0	0	0	0	0	0	0	0
(b)	255	255	0	60	100	100	0	0	1
(c)	0	0	127.5	240	100	50	1	1	0
(d)	255	255	255	0	0	100	0	0	0
(e)	0	255	255	180	100	100	1	0	0
(f)	255	0	0	0	100	100	0	1	1
(g)	255	255	255	180	0	100	0	0	0
(h)	0	0	0	300	50	0	0	0	1

2. Given a color $c_1 = \text{RGB}(25, 127, 76)$, which color c_2 can you add to get a color pink? Define (using the RGB model) the pink you are aiming and then the color you added.

Solution. The color pink I aimed for is $\text{RGB}(255, 192, 203)$. I got c_2 as $\text{RGB}(230, 65, 127)$

3. Sketch the color models for RGB (cube) and HSV (cone). In both model sketches, mark (with color name) the position of each of the following colors: black, white, gray, red, cyan, and yellow.

Solution.



4. Given the color $c_1 = \text{RGB}(255, 255, 0)$, if you decrease the "saturation" as defined in HSV until fully desaturated, describe the range of RGB colors you will pass through, as well as the final result.

Solution. *The range of colors you will pass through are from pure yellow to white. In RGB only the Blue value changes from 0 to 255 as saturation is decreased from 100 to 0. The final result is $\text{RGB}(0, 0, 0)$ or $\text{HSV}(60, 0, 100)$ or the color white.*

5. Linearly interpolate the color halfway between red and blue in each of the following color models. (Requires a separate calculation for each subproblem, not just converting the resulting color.)

(a) RGB

Solution. *Interpolation of $\text{RGB}(255, 0, 0)$ and $\text{RGB}(0, 0, 255) = \text{RGB}(128, 0, 128)$*

(b) HSV

Solution. *Interpolation of $\text{HSV}(0, 100, 100)$ and $\text{HSV}(240, 100, 100) = \text{HSV}(120, 100, 100)$*

(c) CMY

Solution. *Interpolation of $\text{CMY}(0, 100, 100)$ and $\text{CMY}(100, 100, 0) = \text{CMY}(50, 100, 50)$*

6. Considering the results of Question 5. What can be concluded about linear interpolation in different color spaces?

Solution. *The linear interpolation of the color spaces are not the same across the different color spaces. CMY and RGB are similar when it comes to linear interpolation but HSV is a different animal. The values of HSV are spatial using angles rather than representational values that work linearly.*

7. Consider the following javascript snippet to setup a vertex buffer and the the pair of vertex/fragment shaders defined in GLSL:

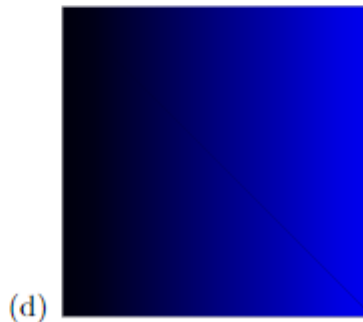
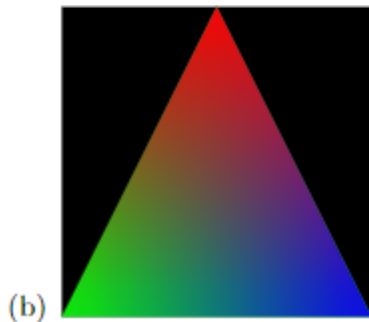
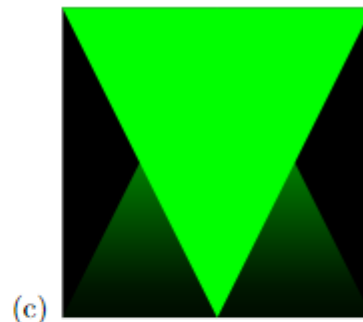
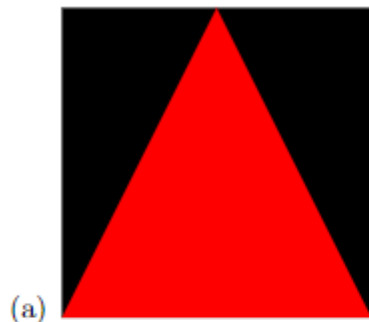
```
1      // === VERTEX SHADER
2      attribute vec2 a_Position;
3      attribute vec3 a_Color;
4      varying vec3 v_Color;
5      main() {
6          v_Color = a_Color;
7          gl_Position = vec4(a_Position, 0.0, 1.0);
8      }
9      // === FRAGMENT SHADER
10     varying vec3 v_Color;
11     main() {
12         gl_FragColor = vec4(v_Color, 1.0);
13     }
14     // === VERTEX BUFFER SETUP IN JAVASCRIPT
15     gl.clearColor(0.0, 0.0, 0.0, 1.0);
16     gl.clear(gl.COLOR_BUFFER_BIT);
17     let vertices = new Float32Array([ __ ]);
18     let a_Position = gl.getAttribLocation(gl.program, "a_Position");
19     gl.vertexAttribPointer(a_Position, __, gl.FLOAT, false, __, __ );
20     gl.enableVertexAttribArray(a_Position);
21     let a_Color = gl.getAttribLocation(gl.program, "a_Color");
```

```

22     gl.vertexAttribPointer(a_Color, __, gl.FLOAT, false, __, __ );
23     gl.enableVertexAttribArray(a_Color);
24     gl.bufferData(gl.ARRAY_BUFFER, vertices, gl.STATIC_DRAW);
25     gl.drawArrays(gl.TRIANGLES, 0, __ );

```

Note there are blanks in this code highlighted in red. For each of the images below, fill the blanks of the code that would generate the output images:



(a)

Solution. Line 17: 0, 0, 0, 255, 0, 0, 0.5, 1, 0, 255, 0, 0, 0.5, 0, 0, 255, 0, 0, 1, 0, 0, 255, 0, 0, 0.5, 1, 0, 255, 0, 0, 0.5, 0, 0, 255, 0, 0
Line 19: 3, 6 * *Float32Array.BYTES_PER_ELEMENT*, 0
Line 22: 3, 6 * *Float32Array.BYTES_PER_ELEMENT*, 3 * *Float32Array.BYTES_PER_ELEMENT*
Line 25: *vertices.length* / 6

(b)

Solution. Line 17: 0, 0, 0, 0, 255, 0, 0.5, 1, 0, 255, 0, 0, 0.5, 0, 0, 0, 128, 128, 1, 0, 0, 0, 0, 255, 0.5, 1, 0, 255, 0, 0, 0.5, 0, 0, 128, 0, 128
Line 19: 3, 6 * *Float32Array.BYTES_PER_ELEMENT*, 0
Line 22: 3, 6 * *Float32Array.BYTES_PER_ELEMENT*, 3 * *Float32Array.BYTES_PER_ELEMENT*
Line 25: *vertices.length* / 6

(c)

Solution. Line 17: 0, 0, 0, 255, 255, 255, 0.25, 0.5, 0, 128, 255, 128, 0.5, 0, 0, 255, 255, 255, 0, 1, 0, 0, 255, 0, 0.5, 0, 0, 0, 255, 0, 1, 1, 0, 0, 255, 0, 1, 0, 0, 255, 255, 255, 0.75, 0.5, 0, 128, 255, 128, 0.5, 0, 0, 255, 255, 255

Line 19: 3, 6 * *Float32Array.BYTES_PER_ELEMENT*, 0

Line 22: 3, 6 * *Float32Array.BYTES_PER_ELEMENT*, 3 * *Float32Array.BYTES_PER_ELEMENT*

Line 25: *vertices.length* / 6

(d)

Solution. Line 17: 0, 0, 0, 255, 255, 255, 0, 1, 0, 255, 255, 255, 1, 1, 0, 0, 0, 255, 0, 0, 0, 255, 255, 255, 1, 0, 0, 0, 0, 255, 1, 1, 0, 0, 0, 255

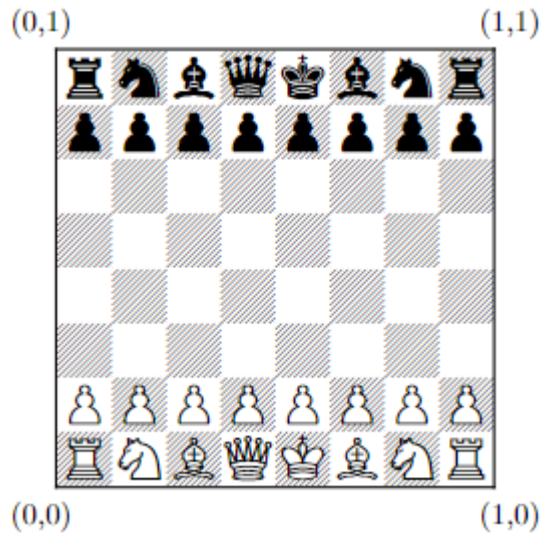
Line 19: 3, 6 * *Float32Array.BYTES_PER_ELEMENT*, 0

Line 22: 3, 6 * *Float32Array.BYTES_PER_ELEMENT*, 3 * *Float32Array.BYTES_PER_ELEMENT*

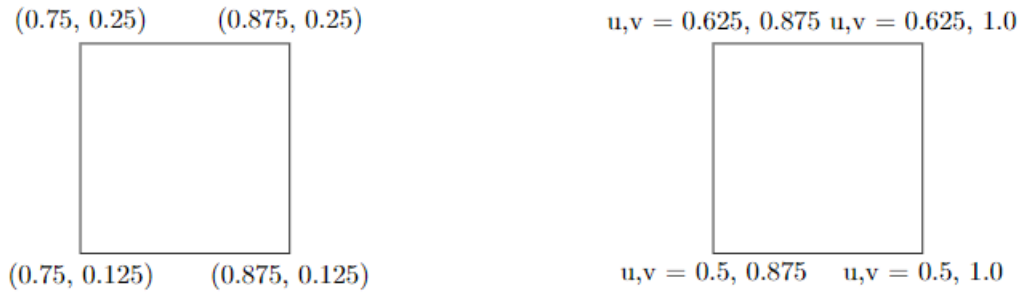
Line 25: *vertices.length* / 6

Textures

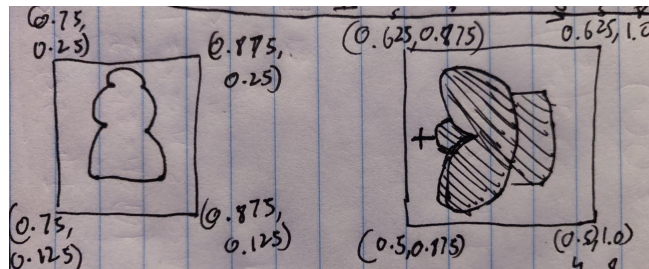
1. Considering the following chessboard texture with (u,v) coordinates:



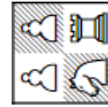
- (a) Draw the approximate mapping on each quad if they were textured using the above image.



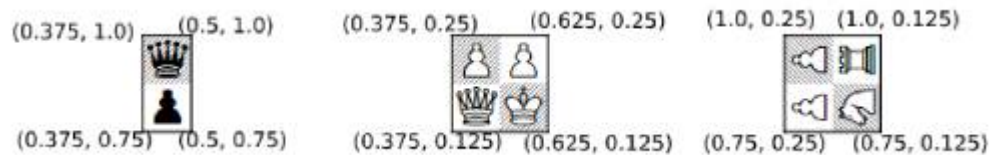
Solution.



- (b) Label the (u,v) texture coordinates for each of the 4 vertices that would result in the image below (just write them next to the corner of each square).



Solution.



2. Describe how and when one could use Spherical Parameterization to map a texture onto a 3D model.

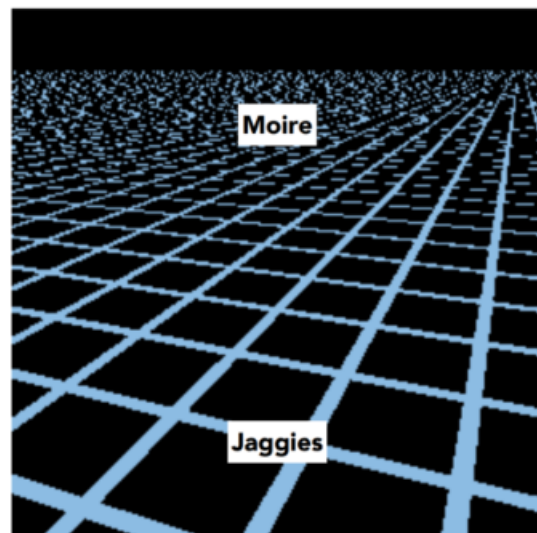
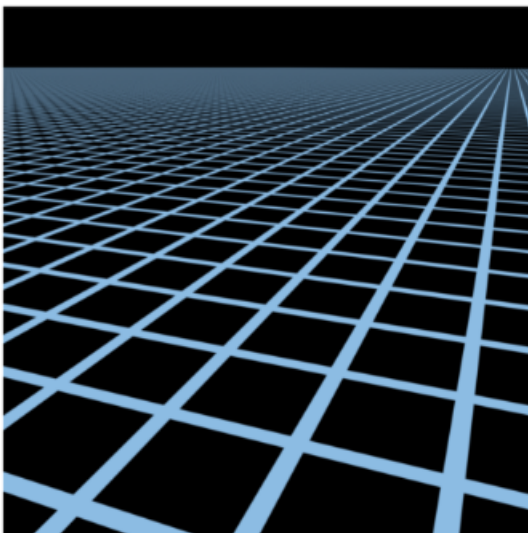
Solution. *Spherical Parameterization is a way to texture map a 3D object that is spherical in nature. Rather than making a 2D rectangular texture map a spherical parameterization can help with minimizing duplicated or missing data.*

We can use spherical parameterization on 3D objects that are spherical in nature.

3. In texture mapping, what is the difference between magnification and minification?

Solution. *Magnification is used when the fragment in the texture space is smaller than a texel. Minification is used when the area of the fragment in texture space is larger than the texel.*

4. We textured a floor in our world and wish it looked like the left image, instead we got the right image, with speckled Moire patterns in the back and Jaggies in the front.



- (a) Explain why these Moire patterns appear and how to fix them.

Solution. *The reason for Moire patterns is because you are skipping texture data. Basically there is not enough space to render the texture for the spatial point.*

We can solve this by using MipMapping and rendering smaller versions of the texture so that they have space to fit without pixels skipping texel data.

- (b) Explain why these Jaggies patterns appear and how to fix them.

Solution. *The reason for the Jaggies is because you are trying to pull more pixel data from the same texel and to solve that use nearest neighbor but that ends up making the jaggies as you see. By using a Magnification method (bilinear/trilinear or bicubic) we can fix the jaggies.*

5. What is the following fragment shader doing?

```
1     varying vec2 uv;
2
3     uniform sampler2D tex;
4     uniform vec4 baseColor;
5     void main() {
6         vec4 texColor = texture2D(tex, uv);
7         gl_FragColor = 0.5 * texColor + 0.5 * baseColor;
8     }
```

Solution. *The fragment shader here is retrieving the texel data from the texture loaded and the vector uv is to specify the texel data location from the 2D texture map. Then it assigns the texel data to the color of the fragment after transforming it in half and adding it to the base color.*

Reading Q's Ch5

1. What is the difference between using multiple buffers to store vertex data vs. interleaving vertex data in one buffer?

Solution. *Using multiple buffers to store vertex data is great for small amounts of data but for large datasets with many many vertices it is better for the storage and memory to store the multiple buffers interleaved in a single buffer. It saves space and reduces the load on the processor to parse the multiple buffers instead the GPU will do that job instead. The single buffer therefore frees up resources for the other parts of the program.*

2. Describe the two process that take place between the vertex and fragment shaders.

Solution. *The two processes that take place between the vertex and fragment shaders are the Shape Assembly task and the Rasterization task. Shape Assembly takes the vertices and starts assembling the shape geometrically and Rasterization breaks that geometric shape into multiple fragments to be addressed by the fragment shaders.*

3. In the context of the fragment shader, describe how varying variables can be used to interpolate data among fragments.

Solution. *Since we have the vertices and data assembled geometrically the fragment shader will get the interpolated values from the rasterization process from the data in the vertex shader's varying variables as they are designed to send data from the vertex shader to the fragment shader.*

4. In the context of texture mapping, what is a magnification method? Enumerate and explain different methods.

Solution. *The magnification method is upsampling, meaning that you map the texture to an area larger than the size of the texture.*

(a) Nearest Neighbor - Uses the value of the nearest texel to the pixel being textured

(b) Bilinear - Interpolates the pixel value using known vertex values

5. In the context of texture mapping, what is a minification method? Enumerate and explain different methods.

Solution. *The minification method is downsampling, meaning that you map the texture to an area smaller than the size of the texture.*

(a) MipMapping - using a pyramid of successively smaller versions of the original texture.

Notes: Reading Ch5

1. Varying variables pass data from vertex shaders to fragment shaders through rasterization
2. Single buffer for the vertex and fragment shader data is often more efficient on the CPU since most of the parsing workload is passed off to the GPU.
3. Multiple Buffers are good for small vertex counts
4. Single buffer with a stride and offset can easily parse through interleaved vertex and fragment data and use the varying variable to pipeline efficiently
5. textures use coordinates in terms of (u,v) to locate texel data for each pixel
6. Minification and Magnification are necessary to the rendering of texture to make sure there is no Mip or Jaggy artifacts
7. Mipmapping for minification and nearest neighbor or bilinear/trilinear or bicubic for magnification.